


# Scalable Distributed Computing - 2026/27

## Lecture 8 - Performance Measurement and Empirical Performance Analysis II

Patrizio Dazzi patrizio.dazzi@unipi.it

ACT IV

# Measuring performance correctly

Comparable operating points require a consistent measurement procedure.

# Do not measure every moment in the same way

A performance run often contains different phases:

start-up → settling → measurement

During start-up, caches, connections, runtimes, buffers, and other system state may still be evolving.

**Define explicitly when measurement begins and ends.**

# Warm-up must be justified by observed behavior

It is tempting to say:

"ignore the first 30 seconds."

But why 30?

Instead, inspect the evolution of relevant measurements over time.

time $\longrightarrow$ observed performance

The purpose is to identify whether the system is still undergoing a systematic transition.

**Discarding an initial interval is justified by system behavior, not by convention.**

# A stable operating region need not be perfectly constant

A system under a fixed workload may still exhibit variability.

We are not looking for

$$X(t) = \text{constant}.$$

We are looking for a region without an obvious systematic transition such as

continuous growth, continuous decline, unfinished start-up.

**Variability and instability are not the same thing.**

# A measurement window can be too short

Suppose throughput varies during a run.
A 2-second interval may observe

$$X = 8{,}700 \text{ requests/s},$$

while a longer interval over the same operating condition gives

$$X = 8{,}050 \text{ requests/s}.$$

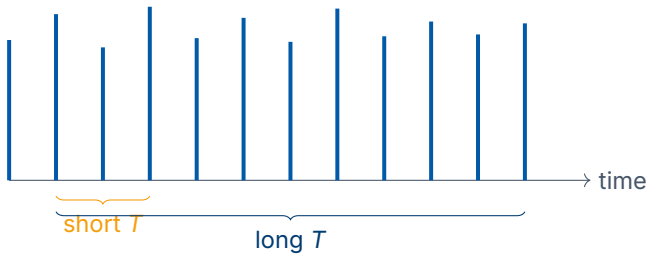
Short windows are more exposed to transient variation.

**The observation interval must be long enough to represent the behavior being studied.**

# A measurement window acts as a temporal filter

The throughput definition already contains the averaging window:

$$X_T(t) = \frac{C(t+T) - C(t)}{T}.$$




Short $T$ preserves fast variation; longer $T$ averages more of that variation away.

**Modelling note.** Choose $T$ to match the behavior the experiment is intended to characterize.

# Repeat the same operating point

Suppose the same experiment produces

$$8{,}120, \quad 7{,}940, \quad 8{,}080, \quad 7{,}990, \quad 8{,}110$$

requests/s across five runs.

The variation itself is information.
A single run would hide it.

**Repeated runs reveal whether an observed result is reproducible.**

# Repeated runs can be summarized quantitatively

For repeated measurements $X_1, \ldots, X_m$,

$$\overline{X} = \frac{1}{m}\sum_{j=1}^{m} X_j, \qquad s_X = \sqrt{\frac{1}{m-1}\sum_{j=1}^{m}(X_j - \overline{X})^2}.$$

The first quantity summarizes the center of the repeated measurements; the second summarizes their run-to-run spread.

For the five runs just shown,

$$\boxed{\overline{X} = 8{,}048 \text{ requests/s}, \qquad s_X \approx 79 \text{ requests/s}}.$$

**Report repeatability quantitatively rather than relying on a single run.**

# Worked example: compute deviations from the repeated-run mean

For the five runs,

$$x_j \in \{8120, 7940, 8080, 7990, 8110\}, \qquad \overline{x} = 8048.$$

The deviations are

$$x_j - \overline{x} \in \{72, -108, 32, -58, 62\}.$$

Squaring them gives

$$5184,\ 11664,\ 1024,\ 3364,\ 3844,$$

whose sum is

$$\boxed{\sum_{j=1}^{5} (x_j - \overline{x})^2 = 25{,}080}.$$

# Worked example: compute standard deviation and relative spread

The sample standard deviation is

$$s_X = \sqrt{\frac{1}{5-1}\sum_{j=1}^{5}(X_j - \overline{X})^2}.$$

Substituting the previous result,

$$s_X = \sqrt{\frac{25{,}080}{4}} = \sqrt{6{,}270} \approx 79.2 \text{ requests/s}.$$

Relative to the mean,

$$\boxed{\frac{s_X}{\overline{X}} = \frac{79.2}{8048} \approx 0.0098 \approx 0.98\%}.$$

**Absolute and relative spread provide complementary views of repeatability.**

# Aggregation can hide structure

Suppose two runs have the same average throughput:

$$\overline{X} = 8{,}000 \text{ requests/s.}$$

One is approximately stable.
The other alternates between

$$5{,}000 \quad \text{and} \quad 11{,}000 \text{ requests/s.}$$

The average alone cannot distinguish them.

**Inspect temporal behavior before reducing measurements to summary statistics.**

# Do not remove inconvenient measurements

Suppose one run produces an unexpected point.
There are at least two possibilities:

measurement problem or real system behavior.

Removing the point because it “looks wrong” destroys evidence.
Instead, investigate whether there is a documented reason to exclude it.

**Data cleaning is part of the methodology and must be explainable.**

# A single experiment may cross different operating regimes

As workload increases, a previously irrelevant limitation may become active.
Then measurements may show an abrupt change in behavior.

regime A ⟶ change ⟶ regime B.

This is not automatically noise.
It may indicate that the system is now operating under different constraints.

**Do not force one interpretation across an observed change of regime.**

ACT V

# From question to evidence

A performance result is only as meaningful as the path that produced it.

# Turn a performance statement into a claim with evidence

Compare two statements:

**Vague**

"The service can handle about 6000 requests/s."

**Evidence-scoped**

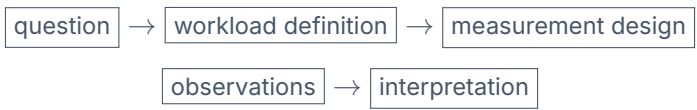
Under workload $W$ and configuration $S$, the tested point $r_{\text{in}} = 6000$ requests/s satisfies $p_{95} \leq 60$ ms across the specified measurement procedure.

A useful result records the chain

claim ⟵ observations ⟵ measurement design ⟵ workload + configuration.

**The strength of a conclusion cannot exceed the scope of the evidence that supports it.**

# From a performance question to an interpretable result




At every step, ambiguity propagates forward.

**Good performance analysis begins before the first measurement is collected.**

# What should a performance result report?

At minimum:

workload | system configuration

measurement interval | performance metrics

And, when relevant:

generation method | repetitions | data treatment.

**A performance number without its context is not reproducible evidence.**

# Do not confuse three different questions

1. What workload is presented to the system?
2. How does the system respond?
3. Why does the system behave that way?

This lecture has addressed the first two.
The third requires additional models and system knowledge.

**Measurement describes behavior; explanation requires a model.**

# Models and measurements play different roles

A model lets us simplify a system enough to reason about it. A measurement tells us whether the resulting claim is supported by evidence.

When we use a model to interpret measurements, keep four questions visible:

- What does it preserve?
- What does it discard?
- What can I derive?
- How do I validate it?

observe → abstract → model → derive → measure → revise

**A model is an instrument for reasoning, not a substitute for evidence.**

# What we can now do

We can now:

- measure throughput and latency without confusing input and output rates;
- describe latency empirically through distributions and percentiles;
- interpret throughput and tail latency together across load levels;
- identify measured feasible operating points under explicit constraints;
- design measurement windows and repeated runs that support reproducible evidence;
- avoid turning measurements into explanations without an explicit model.

**A later step is to explain how load, waiting, and limited service capacity interact.**

# Selected references for further study


- D. J. Lilja, *Measuring Computer Performance: A Practitioner's Guide*, Cambridge University Press, 2000.
- R. Jain, *The Art of Computer Systems Performance Analysis*, Wiley, 1991.
- M. Harchol-Balter, *Performance Modeling and Design of Computer Systems*, Cambridge University Press, 2013.
- J. Dean and L. A. Barroso, "The Tail at Scale," *Communications of the ACM*, 56(2), 74–80, 2013.


**Performance evidence starts with careful measurement and uses models only with explicit assumptions and scope.**

# Q & A

*Any questions?*